

Unit 2: Task Parallel Library & async/await

강사: 조영민

목표

- Task와 Thread의 차이를 이해한다
- `async/await` 패턴으로 비동기 코드를 작성할 수 있다
- `CancellationToken`을 사용한 협력적 취소를 구현할 수 있다
- `Channel`을 활용한 비동기 Producer-Consumer 파이프라인을 설계할 수 있다
- `ContinueWith`와 `await`의 예외 처리 차이를 설명할 수 있다
- `ValueTask`와 `Task`의 성능 차이를 벤치마크로 확인할 수 있다

Task란? -- Thread와의 차이 (1/2)

비교 항목	Thread	Task
추상화 수준	OS 스레드를 직접 생성	비동기 작업의 핸들 (스레드를 직접 만들지 않음)
생성 비용	높음 (스택 할당 ~1MB)	낮음 (ThreadPool 재사용)
반환값	없음	<code>Task<T></code> 로 결과 반환 가능

Task란? -- Thread와의 차이 (2/2)

비교 항목	Thread	Task
예외 전파	해당 스레드에서만 발생	await 시 호출 측으로 전파
취소	직접 플래그 관리	CancellationToken 표준 패턴
조합	수동 Join	WhenAll , WhenAny 조합

면접 TIP: "Task는 스레드가 아닙니다. 비동기 작업의 상태와 결과를 추적하는 핸들입니다."

async/await 기본 -- 비동기의 핵심

- **async**: 메서드에서 **await** 사용을 가능하게 하는 한정자
- **await**: 비동기 작업 완료까지 **스레드 블로킹 없이** 대기
- **Task.Delay**: `Thread.Sleep` 의 비동기 버전 (스레드를 블로킹하지 않음)

```
static async Task FakeDownloadAsync(string name, int ms)
{
    Console.WriteLine($"{name} 시작");
    await Task.Delay(ms);
    Console.WriteLine($"{name} 완료 ({ms} ms)");
}
```

Task.WhenAll -- 병렬 실행 패턴

await 없이 Task를 호출하면 즉시 시작된다. 이를 활용해 여러 작업을 동시에 실행한다.

```
Task task1 = FakeDownloadAsync("파일 A", 1500);  
Task task2 = FakeDownloadAsync("파일 B", 1000);  
Task task3 = FakeDownloadAsync("파일 C", 2000);  
await Task.WhenAll(task1, task2, task3);
```

Task.WhenAll -- 성능 비교

패턴	총 소요 시간
순차 실행 (각각 await)	$1500 + 1000 + 2000 = 4500\text{ms}$
병렬 실행 (WhenAll)	$\max(1500, 1000, 2000) = 2000\text{ms}$

핵심: WhenAll은 모든 작업이 완료될 때까지 대기하며, 총 소요 시간은 **가장 긴 작업** 기준이다.

CancellationToken -- 협력적 취소 패턴

- CancellationTokenSource: 취소 신호를 발생시키는 객체
- CancellationToken: 취소 여부를 감지하는 읽기 전용 토큰
- **협력적 취소**: 런타임이 강제 중단하는 것이 아니라, 작업이 스스로 토큰을 확인하고 응하는 방식

```
static async Task SlowOperationAsync(CancellationToken token)
{
    for (int i = 0; i < 10; i++)
    {
        token.ThrowIfCancellationRequested();
        await Task.Delay(1000, token);
    }
}
```


CancellationToken -- 면접 포인트

면접 TIP: "협력적 취소는 안전한 정리(cleanup)가 가능하다는 장점이 있습니다."

- 런타임이 강제로 스레드를 중단하지 않으므로, **리소스 정리 코드**가 확실히 실행된다
- `ThrowIfCancellationRequested()` 는 `OperationCanceledException` 을 발생시킨다
- `Task.Delay` , `Stream.ReadAsync` 등 대부분의 비동기 API가 토큰을 지원한다

Task.WhenAny -- 타임아웃 패턴

WhenAny는 넘긴 Task 중 가장 먼저 완료된 Task를 반환한다.

```
using var workCts = new CancellationTokenSource();
Task slowWork     = SlowOperationAsync(workCts.Token);
Task timeoutTask  = Task.Delay(TimeSpan.FromSeconds(5));
Task completed    = await Task.WhenAny(slowWork, timeoutTask);
if (completed == slowWork && slowWork.IsCompletedSuccessfully)
    Console.WriteLine("작업이 타임아웃 전에 완료!");
```

핵심: 타임아웃 Task와 작업 Task의 **토큰을 분리**해야 누가 먼저 끝났는지 정확히 판별할 수 있다.

Channel -- 비동기 Producer-Consumer

`System.Threading.Channels`: 비동기 안전한 큐. 생산자와 소비자를 **역할 분리**한다.

```
var ch = Channel.CreateBounded<int>(10);  
// 생산자: ChannelWriter만 사용  
await ch.Writer.WriteAsync(item);  
ch.Writer.Complete();  
// 소비자: ChannelReader만 사용  
await foreach (var item in ch.Reader.ReadAllAsync())  
    Process(item);
```

Channel -- 주요 API

메서드	역할
CreateBounded<T>(n)	크기 제한 채널 (Back-pressure 지원)
WriteAsync	비동기 쓰기. 채널이 가득 차면 대기
Complete()	채널 종료 신호. Reader 루프가 정상 종료됨

핵심: Writer와 Reader 인터페이스가 분리되어 있어 역할 경계가 명확하다.

Back-pressure와 Pipeline 패턴

Back-pressure: 소비가 느리면 생산을 자동 지연하여 **메모리 폭증을 방지**하는 메커니즘.

Pipeline 패턴: 데이터 처리를 여러 단계로 나누어 **동시 실행**하는 구조.

```
Producer -> [rawChannel] -> Transformer -> [transCh] -> Consumer
```

```
var rawCh    = Channel.CreateBounded<int>(10);  
var transCh  = Channel.CreateBounded<int>(10);  
var producer = Task.Run(() => ProduceAsync(rawCh.Writer));  
var transformer = Task.Run(() => TransformAsync(rawCh.Reader, transCh.Writer));  
var consumer  = Task.Run(() => ConsumeAsync(transCh.Reader));  
await Task.WhenAll(producer, transformer, consumer);
```

await foreach와 IAsyncEnumerable

await foreach: `IAsyncEnumerable<T>` 을 비동기 순회하는 C# 8.0 문법.

```
await foreach (var item in reader.ReadAllAsync())
{
    int squared = item * item;
    await writer.WriteAsync(squared);
}
```

await foreach -- foreach와 비교

비교	foreach	await foreach
대상	<code>IEnumerable<T></code>	<code>IAsyncEnumerable<T></code>
대기 방식	동기 (블로킹)	비동기 (논블로킹)
사용 시점	컬렉션이 이미 완성	데이터가 점진적으로 도착

핵심: Channel의 `**ReadAllAsync()`와 결합하면 자연스러운 스트리밍 소비가 가능하다.

ContinueWith -- 콜백 스타일 예외 처리

ContinueWith: 콜백 스타일. 예외가 자동 throw되지 않아 **수동 확인** 필요.

```
Task.Run(() => {  
    throw new InvalidOperationException("boom");  
}).ContinueWith(t => {  
    Console.WriteLine($"상태: {t.Status}");  
    Console.WriteLine($"예외: {t.Exception?.InnerException?.Message}");  
});
```


await -- 자연스러운 예외 처리

await: try-catch로 **자연스럽게** 예외를 처리할 수 있다.

```
try
{
    await Task.Run(() => {
        throw new InvalidOperationException("boom");
    });
}
catch (Exception ex) { Console.WriteLine(ex.Message); }
```

핵심: await는 AggregateException을 **자동 언래핑**하여 내부 예외를 바로 throw한다.

AggregateException 언래핑

AggregateException: 여러 예외를 묶는 컨테이너. Task 내부에서 발생한 예외는 항상 이 안에 담긴다.

방식	예외 처리	AggregateException
ContinueWith	<code>t.Exception</code> 으로 수동 확인	직접 접근 필요
await	try-catch로 자동 처리	자동 언래핑 (첫 내부 예외를 throw)

AggregateException -- 코드 비교

```
// ContinueWith: t.Exception은 AggregateException  
t.Exception?.InnerException?.Message // 수동으로 꺼내야 함  
  
// await: AggregateException을 자동 언래핑  
catch (InvalidOperationException ex) // 내부 예외가 바로 잡힘
```

면접 TIP: "await는 AggregateException을 자동 언래핑하여 첫 번째 내부 예외를 throw 합니다."

ValueTask -- Task의 struct 대안

ValueTask: Task의 값 타입(struct) 대안. 이미 완료된 결과를 반환할 때 **힙 할당 없이** 스택에서 처리한다.

```
[Benchmark]  
public Task CompletedTask()      => Task.CompletedTask;  
[Benchmark]  
public ValueTask CompletedValue() => default;
```

ValueTask -- 벤치마크 결과

항목	Task.CompletedTask	default(ValueTask)
타입	참조 타입 (class)	값 타입 (struct)
힙 할당	1회 (싱글톤 재사용)	0회
벤치마크 평균	~0.10 ns	~0.004 ns

- ValueTask는 **이미 완료된 결과를 자주 반환하는 경우** (캐시 히트 등)에 효과적이다
- 비동기 경로가 대부분인 경우에는 Task로 충분하다

async void의 위험성

async void는 이벤트 핸들러 외에는 **절대 사용하지 말 것**.

항목	async Task	async void
예외 전파	호출자에게 전파	SynchronizationContext로 전파 (크래시)
await 가능	가능	불가능
취소 추적	CancellationToken 전달 가능	추적 불가
테스트	용이	곤란

async void -- 코드 비교

```
// 위험: 예외 발생 시 프로세스가 크래시할 수 있음
async void FireAndForget()
{ await Task.Delay(100); throw new Exception(); }

// 안전: 호출자가 예외를 관찰할 수 있음
async Task SafeOperation()
{ await Task.Delay(100); throw new Exception(); }
```

면접 TIP: "async void는 fire-and-forget이 아닙니다. 예외가 프로세스를 종료시킬 수 있습니다."

고찰점 -- 게임 서버에서 async/await

게임 서버에서 async/await를 쓸 때 주의할 점:

- **SynchronizationContext**: 서버 환경에는 UI 스레드가 없다. `ConfigureAwait(false)` 적용을 고려
- **Task.Run 남용 주의**: I/O-bound 작업에는 async I/O를, CPU-bound에만 Task.Run을 사용
- **Channel 활용**: 패킷 수신 -> 처리 -> 응답 파이프라인에 적합
- **CancellationToken**: 클라이언트 접속 종료 시 진행 중인 작업을 안전하게 취소

고찰점 -- 면접 포인트

면접 TIP: "async/await는 스레드를 늘리는 것이 아니라, 기존 스레드를 **효율적으로 재사용**하는 패턴입니다. I/O 대기 중 스레드가 반환되어 다른 요청을 처리할 수 있습니다."

실습 안내 (1/2)

실습	주제	핵심 개념
Lab01	Async Basics	async/await, Task.WhenAll, Task.Delay
Lab02	Cancellation & Timeout	CancellationToken, Task.WhenAny, 협력적 취소
Lab03	Channel Pipeline	Bounded Channel, Back-pressure, await foreach

실습 안내 (2/2)

실습	주제	핵심 개념
Lab04	Continuation & Exception	ContinueWith vs await, AggregateException 언래핑
Lab05	ValueTask Benchmark	ValueTask vs Task, BenchmarkDotNet, 힙 할당 비교

각 Lab의 실험 과제를 수행하며 파라미터를 변경하여 동작 차이를 관찰하세요.

핵심 정리

- Task는 스레드가 아니라 비동기 작업의 핸들이다
- `async/await`는 스레드를 블로킹하지 않고 비동기 대기하는 패턴이다
- `Task.WhenAll`은 병렬 실행, `Task.WhenAny`는 경쟁/타임아웃 패턴에 사용한다
- `CancellationToken`은 협력적 취소의 표준 메커니즘이다
- `Channel`은 비동기 안전한 Producer-Consumer 패턴을 제공하며, Back-pressure를 지원한다
- `await`는 `AggregateException`을 자동 언래핑하여 예외 처리를 단순화한다
- `ValueTask`는 이미 완료된 결과를 자주 반환하는 핫 경로에서 힙 할당을 제거한다
- `async void`는 이벤트 핸들러 외에는 사용하지 않는다